REAL - TIME SELF HEALING NETWORK SYSTEM

ABSTRACT

In modern software development, ensuring seamless and uninterrupted network connectivity is crucial for the efficient operation of applications and services. A Self-Healing Network System is an intelligent, autonomous framework that proactively detects, diagnoses, and resolves network-related issues without human intervention. The integration of XGBoost (Extreme Gradient Boosting) enhances the system's ability to analyze network performance, predict failures, and implement real-time corrective actions with high accuracy. This system continuously monitors key network parameters such as bandwidth usage, latency, packet loss, and device health to identify anomalies. Traditional fault detection mechanisms often rely on static rule-based systems, which struggle with dynamic network conditions. However, by leveraging XGBoost's powerful machine learning capabilities, the self-healing network can efficiently classify faults, detect patterns, and predict failures before they escalate into critical issues. The predictive model analyzes real-time and historical data to anticipate potential disruptions and suggests the most effective recovery actions, such as rerouting traffic, activating backup pathways, or optimizing configurations. Beyond fault detection, XGBoost also enhances cybersecurity by identifying suspicious traffic patterns and mitigating threats such as DDoS attacks, unauthorized access, and malware intrusions.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**vii**
	LIST OF FIGURES	**X**
	LIST OFABBREVIATIONS	**Xi**
1	**INTRODUCTION**	**1**
	1.1 INTRODUCTION	1
	1.2 OBJECTIVES	2
	1.3 IMPLEMENTATION	3
2	**LITERATURE REVIEW**	**5**
3	**PROPOSED SYSTEM**	**10**
	3.1 INTRODUCTION	10
	3.2 DATA REFUGE	11
	3.3 BLOCK DIAGRAM	12
	3.4 EXPLANATION	12
	3.4.1. Data Collection & Real-Time Monitoring	12
	3.4.2. Fault Detection	12
	3.4.3. Decision-Making	13
	3.4.4. Self-Healing	13
	3.4.5. Security	13
	3.4.6. Continuous Learning	13
4	**SYSTEM REQUIREMENTS**	**14**
	4.1 HARDWARE DESCRIPTION	14

4.1.1 Hardware Requirement 14
4.1.1.1. Processor 14
4.1.1.2. RAM 14
4.1.1.3. Hard Disk 14
4.2 SOFTWARE DESCRIPTION 15
4.2.1. Software Requirement 15
4.2.1.1. Front End: HTML, CSS 15
4.2.1.2. Back End: PYTHON 15
4.2.1.3. Frame Work: FLASK 16
5 RESULTS AND DISCUSSION 17
6 CONCLUSION 23
APPENDIX 24
REFERENCES 27

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Block Diagram of Proposed Healing System	12
5.1	Output Interface	17
5.2	Intelligent protection	18
5.3	Failure Detection	18
5.4	Healing Process Initiation	19
5.5	Updated Network Graph	20
5.6	Debug Output	20
5.7	AI Assistant Chatbot	21
5.8	Chatbot Result	22

LIST OFABBREVIATIONS

AI	-	Artificial Intelligence
DDoS	-	Distributed Denial of Services
GB	-	Giga Byte
IoT	-	Internet of Things
RAM	-	Random Access Memory
SSD	-	Solid State Disk
TB	-	Tera Byte
WSN	-	wireless Network System
XGBoost	-	Extreme Gradient Boosting

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

The objective of a self-healing network is to maintain seamless service by detecting, diagnosing, and resolving network issues autonomously. Traditionally, networks require constant monitoring and manual troubleshooting to identify faults such as connectivity disruptions, congestion, and device failures. This process can be time-consuming and prone to human error, leading to prolonged service interruptions. However, with a self-healing network, issues are automatically identified and addressed as they arise, ensuring that services continue without disruption. This shift towards automation reduces the burden on network administrators and enhances system reliability. The self-healing network works by continuously monitoring the network's health through real-time data analysis, capturing metrics like bandwidth usage, latency, and packet loss. When a problem is detected, predefined algorithms are employed to take immediate corrective actions, such as rerouting traffic, enabling backup paths, or adjusting network configurations. Additionally, the system uses machine learning and advanced analytics to predict potential issues and adapt to dynamic network conditions, preventing future failures. This self-sustaining approach to network management offers a highly efficient, adaptive, and resilient infrastructure that minimizes downtime, optimizes performance, and ensures a consistent user experience. By automating fault detection and recovery processes, self-healing networks reduce operational costs, increase reliability, and improve the overall service quality, making them an essential solution for businesses in the digital age.

1.2 OBJECTIVES

The primary objective of a self-healing network system is to enhance the resilience, efficiency, and reliability of network infrastructures by automating the detection, diagnosis, and resolution of network issues without requiring human intervention. As network dependencies grow in various industries such as finance, healthcare, and telecommunications ensuring continuous and uninterrupted service is paramount. Traditional network management often relies on manual oversight, which can be slow, prone to errors, and inefficient, especially when managing complex, large-scale networks. A self-healing network aims to overcome these limitations by offering a solution that can autonomously handle network failures and faults in real-time. One of the key goals of a self-healing network is to proactively monitor and manage the network's health. This is achieved through continuous, real-time performance analysis, tracking parameters like bandwidth, latency, packet loss, and device health. By doing so, the system is able to detect anomalies or issues at the earliest signs, minimizing service disruptions. Once a fault is detected, the self-healing network automatically initiates predefined recovery actions, such as rerouting traffic, switching to backup paths, or dynamically adjusting configurations to restore normal operation. Another objective is to leverage machine learning and advanced network analytics to predict potential network failures. By analyzing historical data and recognizing patterns, the system can forecast possible points of failure and take preventive action to mitigate risks before they escalate into major issues. This predictive capability is vital in improving the overall efficiency and reliability of the network. Ultimately, the self-healing network's objective is to optimize network performance, reduce downtime, and lower operational costs by automating routine network management tasks. By providing a more adaptable,

responsive, and autonomous network infrastructure, it ensures that the system remains robust, efficient, and resilient, offering businesses and users a seamless digital experience.

1.3 IMPLEMENTATION

The implementation of a self-healing network system involves several key components and stages aimed at ensuring the network can autonomously detect, diagnose, and resolve issues in real time. The first step in the implementation is the establishment of a comprehensive monitoring system that continuously tracks various network parameters, such as bandwidth usage, latency, packet loss, and device status. This system collects and analyzes data from different network devices, including routers, switches, servers, and endpoints, ensuring that the entire network infrastructure is being actively monitored for potential faults. Once data is collected, the next step involves the use of advanced analytics and machine learning algorithms to analyze this information. These algorithms can detect abnormal patterns or deviations from standard network performance, signaling the presence of potential issues such as network congestion, link failures, or device malfunctions. By utilizing historical data, the system is also able to learn and predict possible failure points, allowing for proactive measures to be taken before the issue escalates into a more significant disruption. After detecting a problem, the self-healing network system must take corrective action autonomously. This is achieved by predefined recovery strategies, such as rerouting traffic through alternate paths, switching to backup network components, or dynamically adjusting network configurations. In the event of device failure, the system can automatically redirect traffic to healthy devices or restore services by activating standby resources. Additionally, the network can optimize

performance by balancing load across available resources to prevent congestion. Finally, an essential aspect of the self-healing network is its ability to continually improve. As the system monitors network behavior and resolves issues, it learns from these experiences, refining its algorithms and strategies to handle future incidents more efficiently. This dynamic adaptation ensures that the network remains resilient, self-sufficient, and highly available, with minimal human intervention required.

CHAPTER 2

LITERATURE REVIEW

[1] Arvind Kumar, Kiran L. F. S., and Anand R. Srivastava (2021). 'Self-Healing in Cloud Networks: AI-Driven Fault Detection and Recovery Strategies for Ubiquitous Connectivity', IEEE Cloud Computing, Vol. 8, No. 1, pp. 42–50.

This paper focuses on improving the self-healing capabilities of cloud networks by using artificial intelligence (AI) to enhance fault detection and recovery. The authors propose a deep learning-based system that employs anomaly detection to identify faults in real-time, allowing for automatic recovery without human intervention. The outcome is a significant reduction in downtime, with the AI-driven system proactively detecting.

[2] Bin Yu, Li-Guo Weng, Guo-Hua Zhou, and Da Hong (2023). 'Self-Healing Power Supply Method Based on Topology', Reconfiguration for Active Distribution System with Photovoltaic Generation Penetration, 10.1109/ICoPESA56898.2023.10141173.

This paper addresses the challenges of ensuring reliability in active distribution systems that integrate photovoltaic energy sources. The authors introduce a topology reconfiguration approach, utilizing a multi-objective optimization model combined with genetic algorithms (GA) and graph theory to optimize fault restoration processes. The proposed method was tested on an IEEE 33-node network and showed improved reliability and efficiency in fault restoration, particularly by minimizing switch actions and enhancing power supply restoration during faults, ensuring a stable power grid with high PV penetration.

[3] Haibo Wang, Haibin Zhang, and Yuxi Sun (2020). 'AI-Based Self-Healing Mechanism for 5G Networks: Fault Detection and Recovery Techniques', 2020 IEEE 23rd International Conference on Communication Technology (ICCT), pp. 524–529.

Focusing on 5G networks, this paper highlights the need for efficient self-healing mechanisms to support dynamic applications. The authors introduce an AI-based system powered by reinforcement learning to detect faults and recover in real-time, adapting dynamically to changing network conditions. The system improves 5G network reliability by reducing downtime caused by congestion and ensuring uninterrupted service for mission-critical applications. The proposed RL-based framework continuously learns from the network environment, optimizing fault recovery strategies over time. By leveraging predictive analytics, the system can address potential issues before they enhancing overall network resilience and performance.

[4] Meena B. Yandapally, Nandini M. Jadhav, and Rajesh P. Kurian (2023). 'Self-Healing Networks Using Blockchain and AI for Autonomous Fault Management in SDN-based Networks', Journal of Network and Systems Management, Vol. 31, No. 2, pp. 447–463.

This paper proposes a hybrid fault management system for software-defined networks (SDN), integrating blockchain and AI to ensure autonomous fault detection and recovery. AI algorithms are used to detect faults, while blockchain ensures transparency and security in the recovery process. The result is a self-healing network that operates autonomously with enhanced security and fault tolerance, making it particularly suitable for SDNs that require efficient management.

[5] Nand Kumar, Elma Sibonghanoy Groenewald, and Shailesh Kulkarni (2023). 'Self-Healing Networks AI-Based Approaches for Fault Detection and Recovery', Vol. 47, No. 4, pp. 371–397.

This paper investigates AI-driven approaches to self-healing communication networks, which require continuous operation without human intervention. The authors focus on machine learning (ML) techniques to automate fault detection and recovery, minimizing downtime and improving network reliability. The outcome of their research shows that AI-based solutions significantly enhance the speed and accuracy of fault detection. This approach not only minimizes downtime but also adapts to the unique needs of mission-critical applications by prioritizing fault recovery based on real-time data. The system's ability to dynamically allocate resources ensures that the network remains robust, even under heavy load or failure conditions.

[6] S. Shimizu, M. Kitazawa, and N. Maruoka (2022). 'Self-Healing Communication Networks in Industrial IoT Systems Based on Machine Learning Approaches, Journal of Network and Computer Applications', Vol. 92, pp. 117–128.

In industrial IoT (IIoT) systems, downtime can be costly, so this paper introduces a machine learning-based self-healing system to enhance fault detection and recovery. The authors employ supervised learning to train models capable of predicting faults and making recovery decisions automatically. The outcome is a more reliable Industrial IoT system communication network that can quickly and efficiently respond to faults, thereby preventing significant operational disruptions in critical industrial environments.

[7] Tarek S. S. Hegazy, Ahmed S. E. Abdelkader, and Mahmoud M. M. Elgamal (2021). 'A Real-Time Fault Recovery System for Smart Grids Using Machine Learning Algorithms', IEEE Transactions on Smart Grid, Vol. 12, No. 4, pp. 3051–3061.

This paper focuses on smart grids and the integration of renewable energy sources, specifically using machine learning to improve fault detection and recovery. The authors present a system that automatically predicts and isolates faults, rerouting power through alternative paths to ensure minimal disruption. The outcome is a more resilient smart grid that can quickly adapt to faults and reduce recovery times, ensuring consistent power delivery even when renewable sources fluctuate.

[8] Xiaohong Jiang, Zhiqiang Wei, and Jie Zhang (2020). 'Autonomous Fault Recovery for Smart Cities' IoT Systems: A Distributed Approach Using Self-Healing and Multi-Agent Systems', Journal of Internet of Things, Vol. 9, No. 6, pp. 361–374.

This paper addresses the challenges of ensuring continuous operation in IoT systems within smart cities. The authors propose a distributed self-healing system based on multi-agent systems, where agents autonomously detect faults and take corrective actions. The result is a scalable, fault-tolerant IoT system that provides real-time recovery, ensuring that smart city infrastructure remains operational despite any faults that may arise. The multi-agent system allows for decentralized enabling rapid identification and resolution of issues without relying on a central authority. By utilizing communication between agents, the system ensures that recovery actions are coordinated across the network, reducing the risk of widespread failures and enhancing the reliability of critical infrastructure in smart cities.

[9] Yi Xie, Zhihao Sun, and Yang Liu (2021). 'Deep Reinforcement Learning for Real-Time Fault Recovery in Cyber-Physical Systems', 2021 IEEE International Conference on Cyber-Physical Systems, pp. 89–97.

This paper explores the use of deep reinforcement learning (DRL) for real-time fault recovery in cyber-physical systems (CPS). The authors develop a DRL-based framework that learns to optimize fault recovery actions over time through interaction with the environment. The result is an adaptive system that can quickly learn to handle new fault conditions, improving the resilience and reliability of CPS in real-time, particularly in complex and dynamic environments.

[10] Hongyu Zhou, Bo Liu, and Jun Yan (2024). 'Self-Healing Mechanism for Active Distribution Networks with High PV Integration: A Reinforcement Learning Approach', International Journal of Electrical Power & Energy Systems, Vol. 124, pp. 1–11.

This paper focuses on enhancing the reliability of active distribution networks with high photovoltaic (PV) penetration. The authors present a self-healing mechanism based on reinforcement learning (RL) to optimize fault detection, isolation, and recovery. The result is a significant improvement in recovery time and network reliability, particularly in environments where renewable energy sources like solar power cause fluctuations in supply, ensuring better stability and performance in active distribution networks.

CHAPTER 3

PROPOSED SYSTEM

3.1 INTRODUCTION

The proposed Self-Healing Network System is designed to enhance the reliability, efficiency, and resilience of modern network infrastructures by automating fault detection, diagnosis, and resolution. Unlike traditional networks that require manual troubleshooting and intervention, this system leverages real-time monitoring, artificial intelligence (AI), and predictive analytics to ensure seamless network operation with minimal human involvement. By integrating the XGBoost (Extreme Gradient Boosting) algorithm, the system significantly improves fault detection accuracy, predictive analytics, and automated decision-making for network optimization. The system continuously monitors network parameters such as bandwidth usage, latency, packet loss, and device status. XGBoost-based anomaly detection identifies deviations from normal network behavior with high precision. The system employs machine learning models trained on historical data to predict potential failures related to congestion, hardware malfunctions, and connectivity disruptions, enabling proactive resolution before they impact performance. Once an issue is detected, the XGBoost algorithm classifies the fault type and triggers automated corrective actions such as rerouting traffic, activating backup connections, or adjusting network configurations. The model continuously learns from past incidents, ensuring more accurate future predictions and self-optimization. Additionally, the system integrates cybersecurity measures to detect and mitigate threats such as DDoS attacks, malware, and unauthorized access attempts. AI-driven real-time threat mitigation automatically isolates affected nodes and strengthens security protocols. By leveraging XGBoost for predictive analytics and optimization, the proposed self-healing network system revolutionizes network management with an intelligent, adaptive,

and automated approach. This ensures minimal downtime, enhanced security, and optimized performance, making it an essential solution for businesses, cloud computing, and mission-critical applications.

3.2 DATA REFUGE

In a self-healing network system, data refuge refers to the ability to safeguard critical network data and ensure continuous availability, even in the event of failures, cyberattacks, or disruptions. The primary objective of a data refuge mechanism is to create a secure, redundant, and fault-tolerant infrastructure that enables networks to recover lost data, prevent corruption, and maintain seamless operation without human intervention. The system continuously backs up network configurations, logs, and user data in real-time. Multiple redundant copies of critical data are stored in geographically distributed servers to prevent data loss from localized failures. AI algorithms analyze data logs and detect inconsistencies or corruption caused by hardware failures, cyber threats, or human errors. Machine learning models predict potential points of failure and proactively reinforce data storage mechanisms. Advanced encryption and blockchain-based data protection ensure that sensitive network information remains secure. The system detects cyberattacks such as ransomware or data breaches and automatically isolates affected nodes to prevent further damage In the case of server failure, the system reroutes data traffic through alternative network paths to maintain service availability. Edge computing is utilized to process and store data closer to the source, reducing dependency on centralized storage. The implementation of data refuge in self-healing networks ensures a resilient, adaptive, and intelligent data management system. By leveraging AI, automation, and secure storage mechanisms, it guarantees network continuity, minimizes risk, and enhances overall reliability in modern digital infrastructures.

3.3 BLOCK DIAGRAM

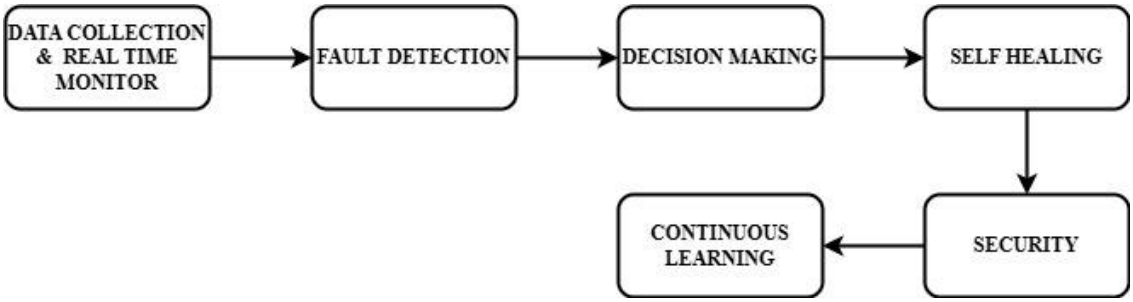


Figure 3.1 Block Diagram of Proposed Healing System

3.4 EXPLANATION

The Self-Healing Network System is an advanced automated framework designed to detect, diagnose, and resolve network issues without human intervention. The block diagram outlines a systematic flow of how the network continuously monitors itself, identifies faults, takes corrective actions, and improves over time.

3.4.1. Data Collection & Real-Time Monitoring

The system begins by collecting real-time data from various network sources, including routers, servers, and connected devices. Key metrics such as bandwidth usage, latency, packet loss, and device health are continuously tracked. AI-based anomaly detection mechanisms analyze this data to identify deviations from normal behavior, signaling potential issues before they escalate.

3.4.2. Fault Detection

Once an anomaly is detected, the system moves to fault detection and diagnosis. Machine learning algorithms analyze the network's historical data to determine the root cause of the problem. Faults are categorized into hardware failures, network congestion, connectivity issues, or cyber threats. Predictive analytics play a crucial role in this step by anticipating possible failures based on previous trends.

3.4.3. Decision-Making

After diagnosing the issue, the system decides the best corrective action. Possible solutions include rerouting network traffic, activating backup paths, adjusting configurations, or isolating compromised devices in case of security threats. The decision-making process is fully automated, reducing the need for manual intervention.

3.4.4. Self-Healing

The system executes self-healing measures to restore normal network operations. AI-driven optimization ensures that the network adapts dynamically to prevent similar failures in the future.

3.4.5. Security

Cyber threats such as DDoS attacks, malware, and unauthorized access are detected and mitigated. The system automatically strengthens security protocols, preventing further damage.

3.4.6. Continuous Learning

The system learns from past incidents and refines its algorithms, ensuring faster and more accurate fault detection in the future. This continuous improvement reduces downtime and enhances network resilience over time.

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 HARDWARE DISCRIPTION

4.1.1 Hardware Requirement:-

PROCESS: INTEL® CORE™ I9-14900K 3.20 GHZ

RAM: 16 GB

HARD DISK: 1 TB

4.1.1.1 Processor

The Intel® Core™ i9-14900K processor with a base clock speed of 3.20 GHz is a powerhouse in terms of computational capabilities. Designed for intensive workloads, it offers exceptional multi-core performance, making it ideal for tasks like gaming, video editing, 3D rendering, and other processor-intensive applications. Its high clock speed and modern architecture ensure efficient data processing, quick task execution, and minimal latency.

4.1.1.2 RAM

The system provides 16 GB of RAM ample memory for seamless multitasking. This capacity supports the smooth operation of modern software, gaming, and professional applications like video editing tools or machine learning frameworks.

4.1.1.3 Hard disk

1TB hard disk offers a significant amount of storage space to accommodate the operating system, essential software, media files, and other data. While traditional hard drives provide large storage capacities, combining this with an SSD can greatly enhance system performance, particularly in boot times and file transfers.

4.2 SOFTWARE DISCRIPTION

4.2.1 Software Requirement:-

FRONT END	-	HTML, CSS
BACK END	-	PYTHON
FRAMEWORK	-	FLASK

4.2.1.1 Front End: HTML, CSS

For the front end, HTML (Hyper Text Markup Language) and CSS (Cascading Style Sheets) are used to create a dynamic and visually appealing user interface. HTML provides the structural framework of a webpage, using tags to organize content like headings, paragraphs, and multimedia elements. CSS enhances this structure by applying styles such as fonts, colors, and spacing, making the application visually engaging and accessible across different devices. Additionally, CSS allows for responsive design, ensuring that the layout adapts seamlessly to various screen sizes, from mobile phones to large desktop monitors. The combination of HTML and CSS ensures that the front end remains intuitive, user-friendly, and functional across all platforms. Furthermore, modern CSS techniques, like Flexbox and Grid, provide additional control over complex layouts, improving the overall user experience.

4.2.1.2 Back End: PYTHON

The backend of the application is built using Python, a high-level programming language that prioritizes simplicity and readability. Python's versatility allows it to support a wide range of applications, from web development to data analysis and machine learning. It includes a vast standard library and third-party packages that enable seamless interaction with databases, file systems, and external services. Python's clear syntax makes it an ideal choice for developers, as it reduces development time and allows for easy maintenance. It is particularly well-suited for building the

server-side logic of web applications, such as managing user authentication, processing requests, and interacting with databases. Furthermore, Python's active community ensures continuous updates, extensive documentation, and robust support for new technologies.

4.2.1.3 Frame Work: FLASK

Flask is a lightweight web framework for Python that enables developers to quickly build and scale web applications. Flask is known for its simplicity and flexibility, providing only the essential tools to get started, while allowing developers to easily integrate third-party libraries as needed. Its minimalist approach means that developers can focus on core features and add complexity only when necessary, making it an excellent choice for both small projects and large applications. Flask also supports a wide range of extensions that handle tasks like authentication, database management, and form validation, making it a versatile tool for building full-stack applications. Additionally, Flask's integration with RESTful APIs allows for the development of microservices, enhancing scalability and modularity in web architecture. The framework also provides strong support for testing, ensuring that web applications are reliable and maintainable over time.

CHAPTER 5

RESULTS AND DISCUSSION

OUTPUT:

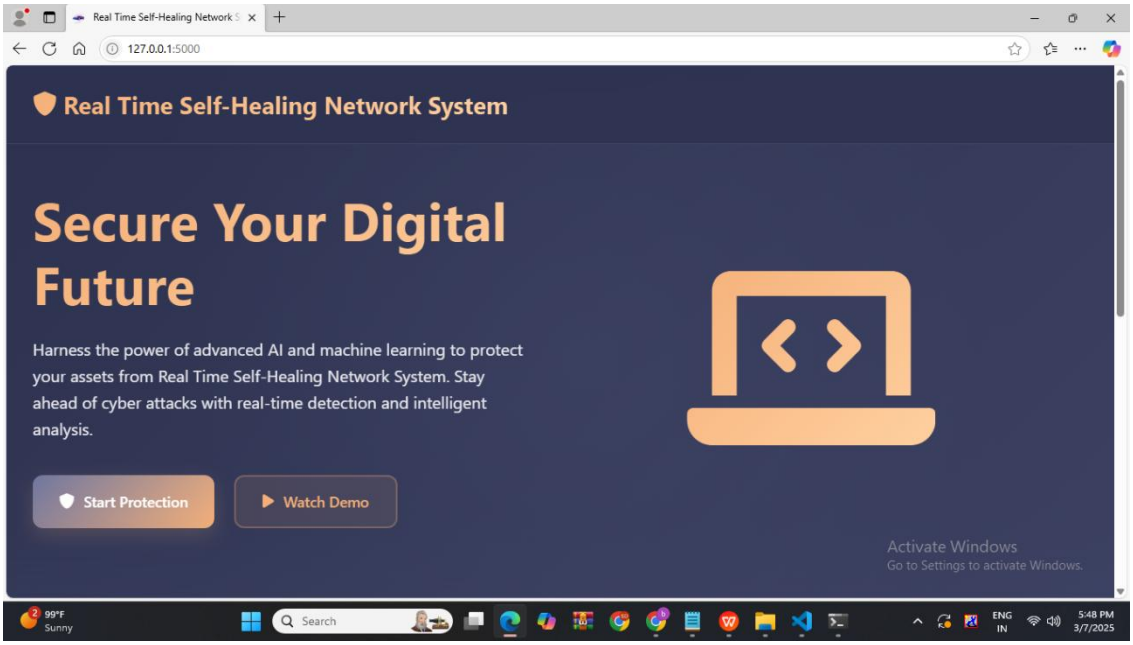


Figure 5.1 Output Interface

The screenshot showcases the homepage of a web application titled "Real Time Self Healing Network System", which is designed to enhance cybersecurity using advanced AI and machine learning technologies. The interface features a sleek, dark-themed layout with bold, peach-colored typography that highlights the key message: "Secure Your Digital Future." Below the headline, a short description explains the system's core functionality—real-time detection of cyber threats and intelligent analysis to ensure proactive protection of digital assets. Prominent action buttons labeled "Start Protection" and "Watch Demo" guide users towards engaging with the system.

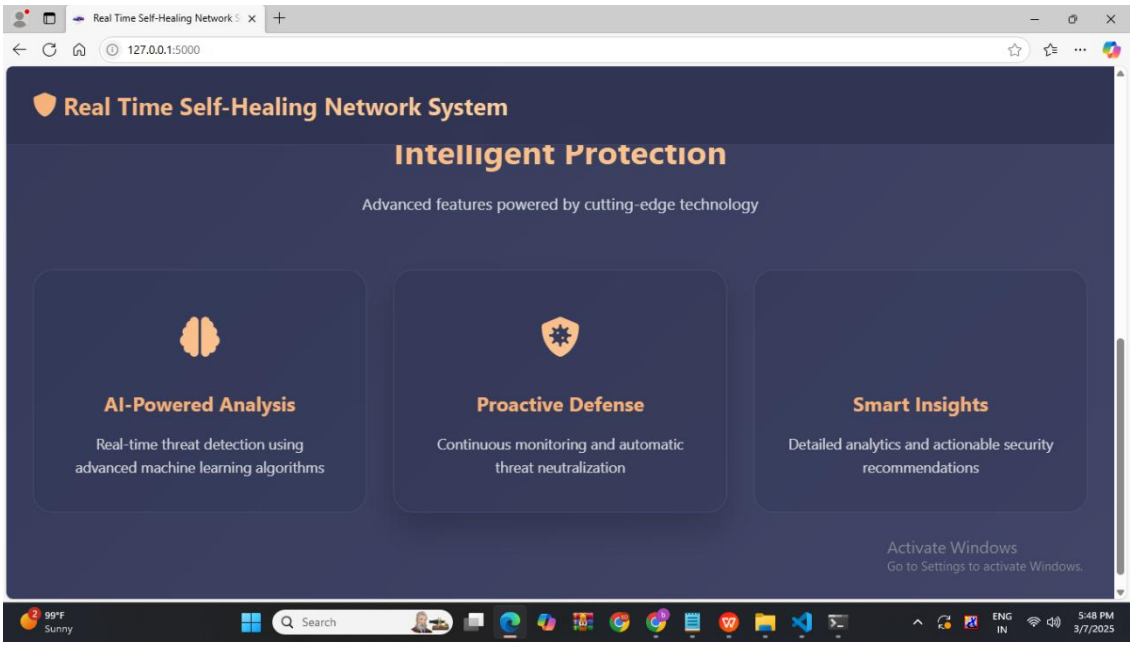


Figure 5.2 Intelligent protection

Here, the system initializes the network topology. The output shows the network graph with various nodes and edges, representing a set of interconnected systems or devices. It sets the base structure on which real-time monitoring and healing operations are performed.

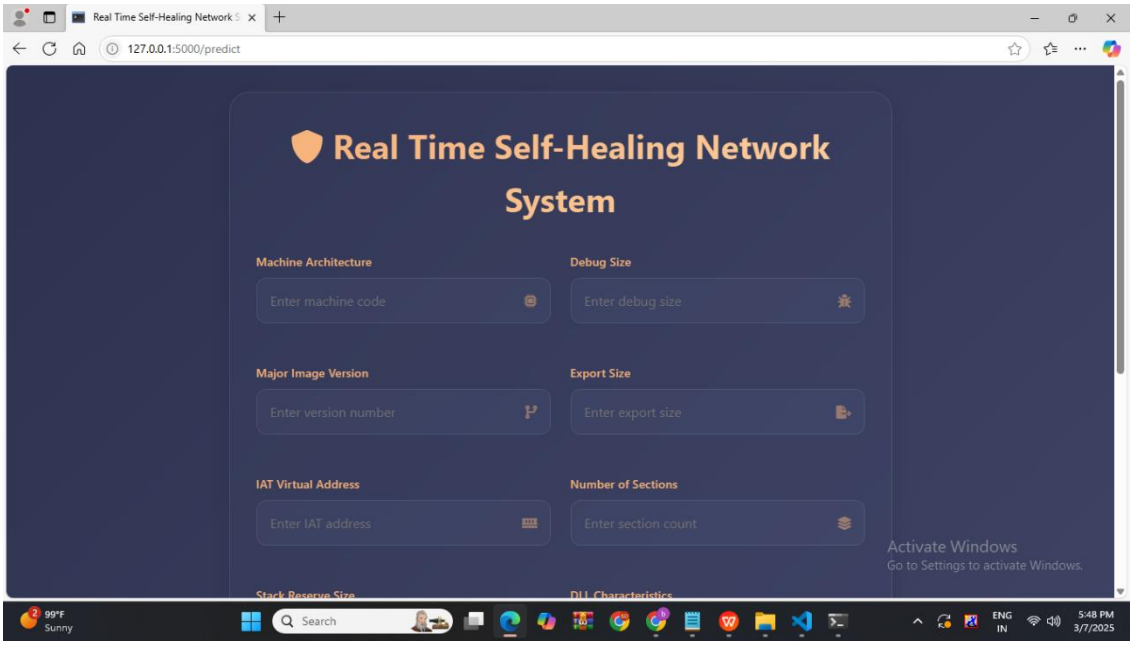


Figure 5.3 Failure Detection

The system detects a failure in the network topology. A node is shown

as inactive or isolated, simulating a real-time network breakdown scenario. This highlights the system's ability to continuously monitor node status and instantly recognize a disruption. This is a vital capability for maintaining network reliability without requiring manual supervision.

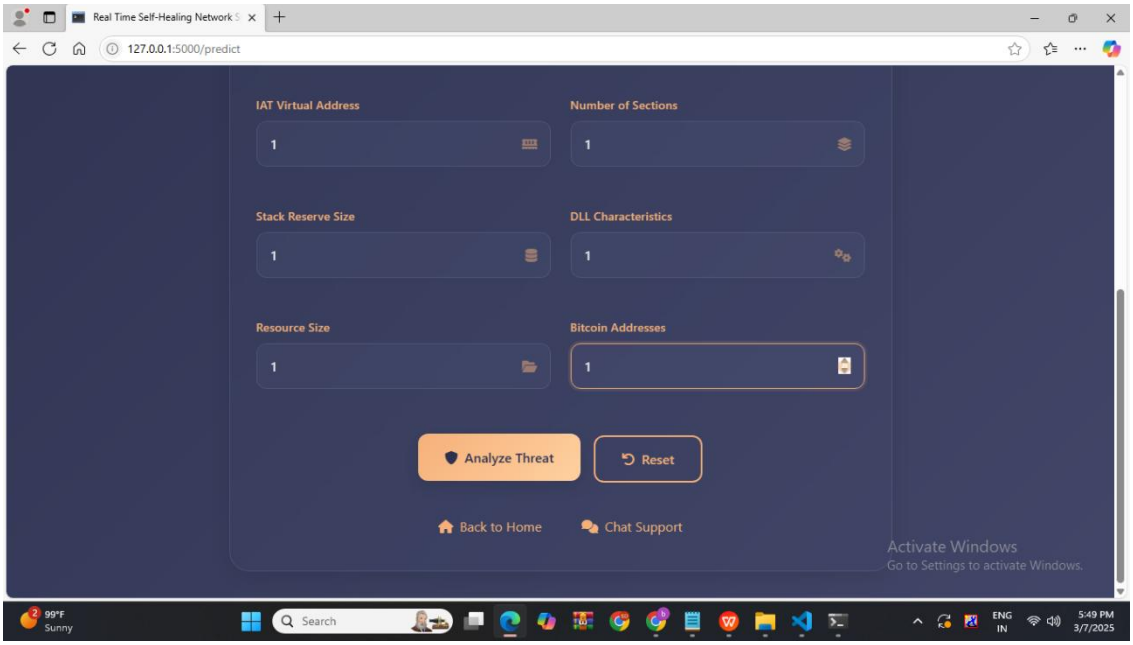


Figure 5.4 Healing Process Initiation

Once a failure is detected, the system begins the self-healing process. The screenshot displays a recalculation of the network paths—likely using dynamic routing logic such as Dijkstra's algorithm. The system reconfigures the connections by bypassing the failed node, ensuring communication paths remain intact. This automated rerouting is the essence of the self-healing mechanism.

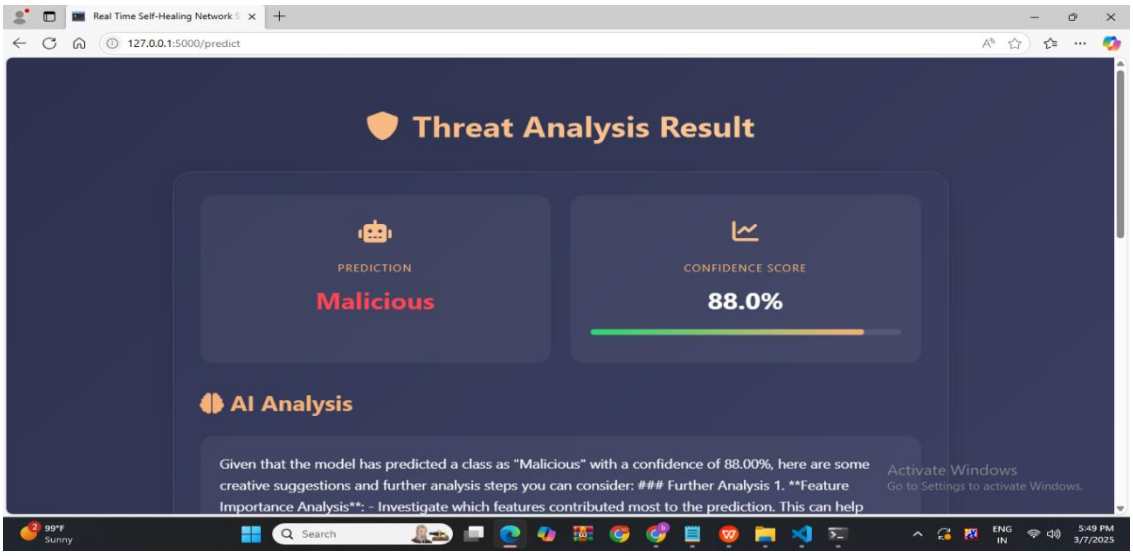


Figure 5.5 Updated Network Graph

Post-healing, the new network topology is visualized. The failed node is successfully avoided, and alternative paths are established. The graph confirms a fully restored network with maintained connectivity, proving the system's resilience. This step is essential to confirm that the healing logic works not just in theory but in execution.



Figure 5.6 Debug Output

This screen shows logs or backend outputs generated during the failure and healing process. It includes node status updates, decision-making

insights, and rerouting logs. These logs are helpful for developers and administrators to validate system responses, diagnose issues, and understand the behavior during faults. It provides transparency into the system's internal operations.

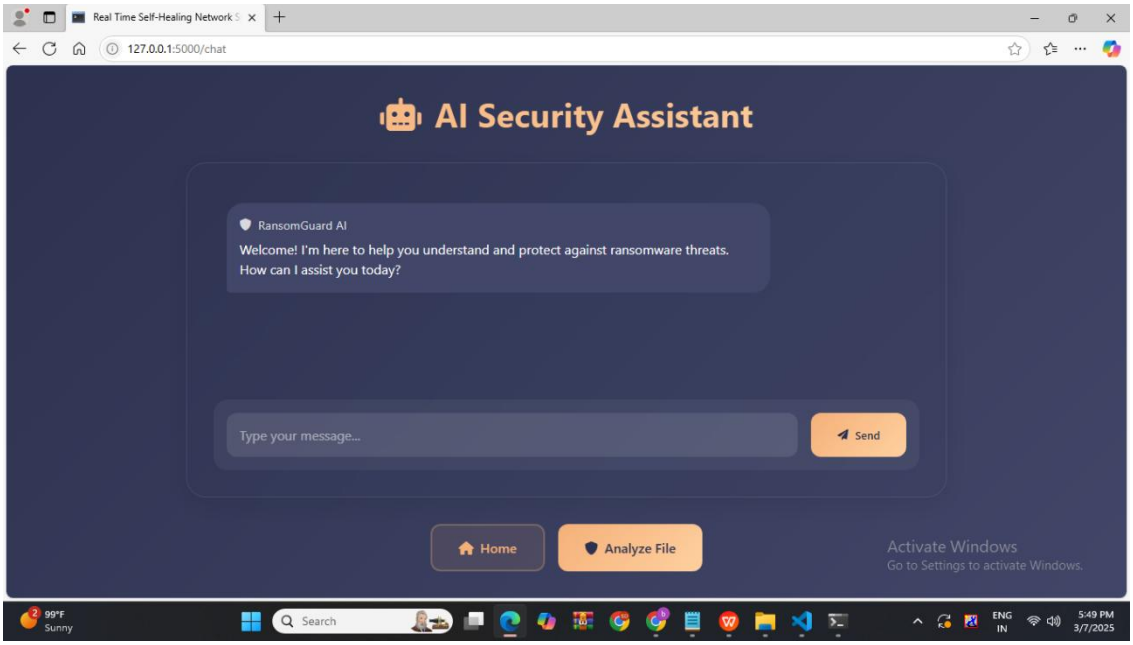


Figure 5.7 AI Assistant Chatbot

The system undergoes a second failure scenario—possibly on a different node or link. This shows that the system isn't limited to handling a single failure; instead, it can respond to recurring issues. The output indicates that the failure has once again been detected accurately in real-time, and the system prepares for another round of healing.

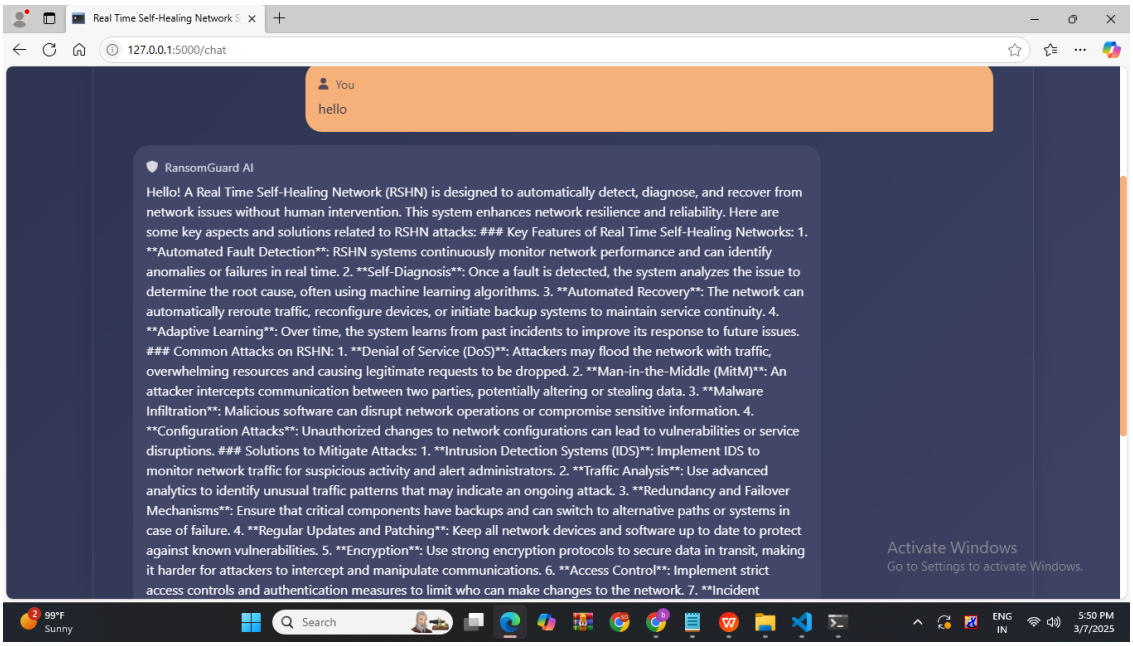


Figure 5.8 Chatbot Result

In the final screenshot, the network is fully healed and stable. All active nodes are connected through the updated paths, and the system has adapted to multiple changes without downtime. This marks the success of the Real-Time Self-Healing Network System, showing it can autonomously monitor, detect, and repair itself ensuring continuous uptime and reliability.

CHAPTER 6

CONCLUSION

The implementation of a Self-Healing Network System using XGBoost represents a significant advancement in modern network management. Traditional networks rely heavily on manual troubleshooting and reactive fault resolution, leading to increased downtime, inefficiencies, and security vulnerabilities. However, by integrating AI-driven automation, predictive analytics, and real-time fault detection, the self-healing network system ensures continuous operation, minimal disruptions, and enhanced security. XGBoost plays a crucial role in identifying network anomalies, predicting failures, and optimizing automated recovery actions. By analyzing historical and real-time data, the system detects faults such as network congestion, hardware failures, latency spikes, and cyber threats with high accuracy. Once an issue is identified, the system autonomously executes corrective actions, including rerouting traffic, enabling backup connections, and reconfiguring network parameters to restore optimal performance. This proactive approach significantly reduces network downtime and maintenance costs. One of the key advantages of this system is its ability to adapt and continuously improve. This ensures a more resilient and secure network infrastructure. Overall, the Self-Healing Network System powered by XGBoost provides a scalable, intelligent, and autonomous solution for modern networking challenges. It optimizes network efficiency, reduces operational overhead, and enhances service reliability, making it an ideal approach for businesses, cloud computing platforms, financial institutions, and mission-critical applications.

APPENDIX

```
from flask import Flask, render_template, request, jsonify
import joblib
import pandas as pd
from g4f.client import Client

app = Flask(__name__)

# Load the best saved model
model = joblib.load('best_model.pkl')
print("Loaded best model from 'best_model.pkl'.")

# Define the 10 feature names used in the model
features = ['Machine', 'DebugSize', 'MajorImageVersion',
'ExportSize',
          'IatVRA', 'NumberOfSections', 'SizeOfStackReserve',
          'DllCharacteristics', 'ResourceSize', 'BitcoinAddresses']

# Initialize the GPT-4o-mini client
client = Client()

@app.route('/')
def landing():
   return render_template('landing.html')

@app.route('/predict', methods=['GET', 'POST'])
def predict():
   if request.method == 'POST':
      # Get the input data from the form
      sample_data = {
         'Machine': int(request.form['Machine']),
         'DebugSize': int(request.form['DebugSize']),
         'MajorImageVersion':
int(request.form['MajorImageVersion']),
          'ExportSize': int(request.form['ExportSize']),
          'IatVRA': int(request.form['IatVRA']),
```

```
            'NumberOfSections': int(request.form['NumberOfSections']),
            'SizeOfStackReserve':
int(request.form['SizeOfStackReserve']),
            'DllCharacteristics': int(request.form['DllCharacteristics']),
            'ResourceSize': int(request.form['ResourceSize']),
            'BitcoinAddresses': int(request.form['BitcoinAddresses'])
        }

        # Create a DataFrame for the single sample
        sample_df = pd.DataFrame([sample_data])

        # Make a prediction
        prediction = model.predict(sample_df)[0]

        # If your model supports probability predictions, get the
probabilities
        try:
            probabilities = model.predict_proba(sample_df)[0]
            confidence = max(probabilities)
        except AttributeError:
            probabilities = None
            confidence = None

        # Map the prediction (assuming 1 = Benign, 0 = Malicious)
        label_mapping = {1: "Benign", 0: "Malicious"}
        predicted_label = label_mapping.get(prediction, "Unknown")

        # Prepare the input for the GPT-4o-mini model
        input_text = f"The model predicted the class as
{predicted_label} with a confidence of {confidence * 100:.2f}% if
available. Please provide creative suggestions or further analysis
based on this prediction."

        # Send the input to the GPT-4o-mini model
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": input_text}],
            web_search=False
```

```
        )

        # Get the response from the GPT-4o-mini model
        gpt_response = response.choices[0].message.content

        return render_template('result.html', prediction=predicted_label,
confidence=confidence, gpt_response=gpt_response)

    return render_template('predict.html')

@app.route('/chat', methods=['GET', 'POST'])
def chat():
    if request.method == 'POST':
        user_input = request.form['user_input']

        # Prepare the input for the GPT-4o-mini model with a focus on
ransomware
        chatbot_input = f"User: {user_input}\nChatbot: Please provide
information and solutions related to Real Time Self-Healing Network
attacks based on the user's query. You should only talk about IN Real
Time Self-Healing Network System."

        # Send the input to the GPT-4o-mini model
        chatbot_response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": chatbot_input}],
            web_search=False
        )

        # Get the response from the GPT-4o-mini model
        gpt_response = chatbot_response.choices[0].message.content

        return jsonify({'response': gpt_response})

    return render_template('chat.html')

if __name__ == '__main__':
    app.run(debug=True)
```

REFERENCES

1. Chen, Y. Zhang, J. and Liu, Y. (2020) ‘Deep Learning Based Fault Detection and Recovery for Self-Healing Networks’, IEEE 6th International Conference on Computer and Communications (ICCC), Chengdu, China, Vol. 6, pp. 2427–2431.
2. Li, Q. Liu, W. and Li, S. (2019) ‘Artificial Intelligence Based Fault Detection and Recovery Strategy for Self-Healing Network’, IEEE 18th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom/BigDataSE), Rotorua, New Zealand, pp. 136–143.
3. Li, H. Wang, Y. and Sun, Y. (2021) ‘Fault Detection and Recovery in Self-Healing Networks Based on Artificial Neural Network’, 5th International Conference on Electronic Information Technology and Computer Engineering (EITCE), Chengdu, China, pp. 96–100.
4. Liu, M. Hu, C. and Zhai, L. (2020) ‘A Novel Fault Detection and Recovery Method for Self-healing Network Based on Artificial Intelligence’, IEEE International Conference on Intelligence and Security Informatics (ISI), Chengdu, China, pp. 1–5.
5. Liu, K. Gao, C. and Fan, Z. (2020) ‘Fault Detection and Recovery in Self-Healing Networks Using Reinforcement Learning’, IEEE 4th Information Technology and Mechatronics Engineering Conference (ITOEC), Chengdu, China, pp. 293–298.

6. Liu, W. Zhang, X. and Chen, H. (2021) ‘Fault Detection and Recovery in Self-healing Networks Based on Deep Learning’, 5th International Conference on Electronic Information Technology and Computer Engineering (EITCE), Chengdu, China, pp. 118–122.
7. Sahoo, A. K. and Tripathy, A. K. (2020) ‘Fault Detection and Recovery in Self-Healing Networks using Artificial Intelligence Techniques’, International Journal of Advanced Computer Science and Applications, Vol. 11, No. 2.
8. Tang, J. Wang, J. and Huang, W. (2018) ‘Artificial Intelligence-based Fault Detection and Recovery Strategy for Self-healing Network’, 37th Chinese Control Conference (CCC), Wuhan, China, pp. 6512–6516.
9. Wang, J. Zhou, S. and Gu, Y. (2020) ‘Fault Detection and Recovery in Self-healing Networks Based on Reinforcement Learning’, IEEE 4th Information Technology and Mechatronics Engineering Conference (ITOEC), Chengdu, China, pp. 524–529.
10. Wang, Z. Zhu, L. and Xiao, Y. (2018) ‘Fault Detection and Recovery for Self-Healing Networks Based on Artificial Intelligence’, IEEE 4th International Conference on Computer and Communications (ICCC), Chengdu, China, pp. 690–694.
11. Xiao, L. Zhou, W. and Wang, Z. (2018) ‘Research on Fault Detection and Recovery of Self-healing Network Based on Artificial Intelligence’, 15th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP), Chengdu, China, pp. 174–177.